

Monish Madhu, Zohaib Sheikh

ECE 316 Lab 5: Stopwatch/Timer

a. HLSM Describing the system

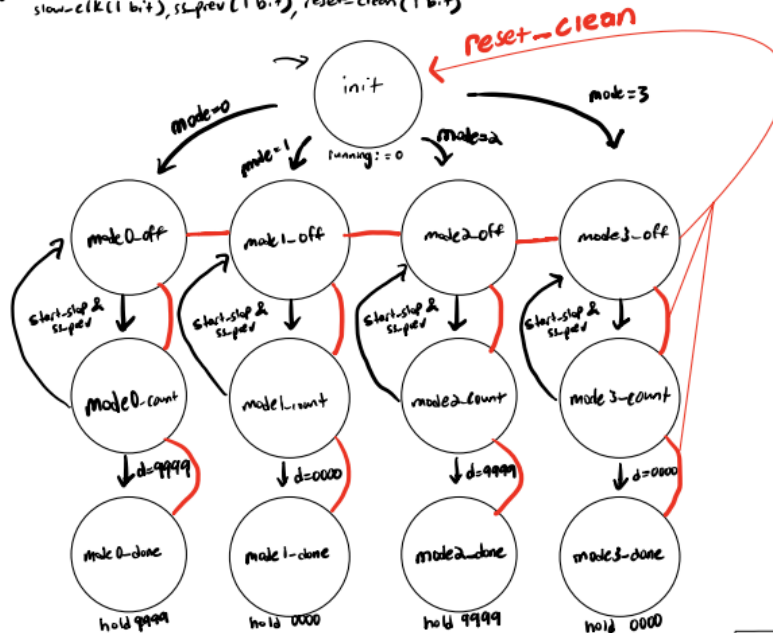
HLSM

inputs: clk, rst, start-stop, mode[1:0], time_value[7:0]

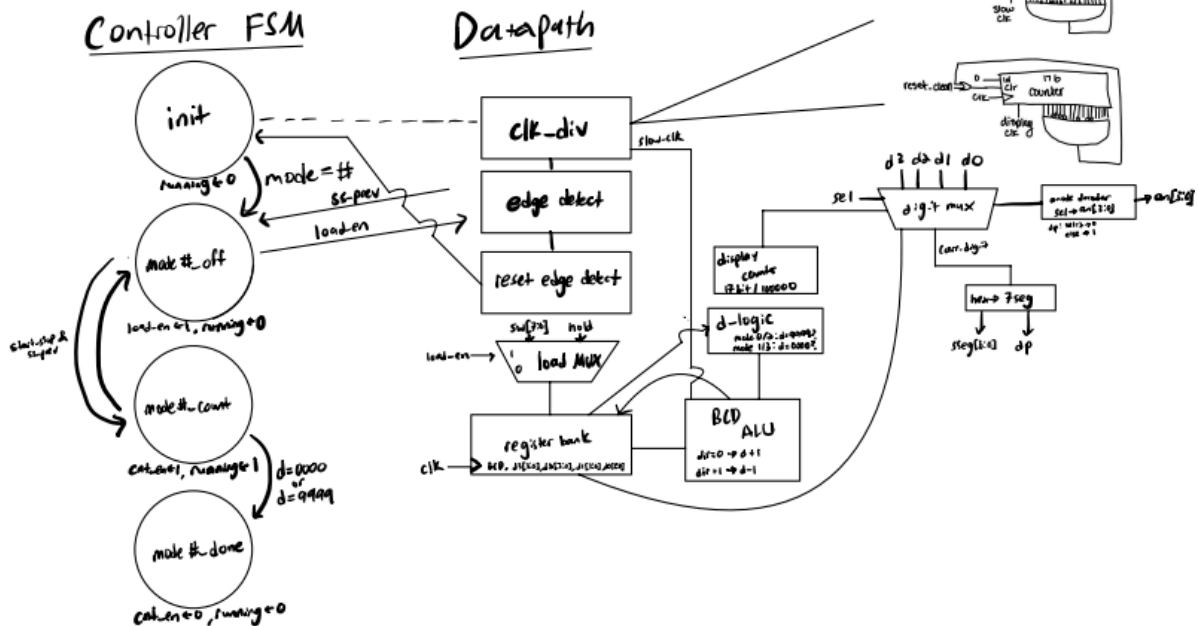
outputs: sseg[6:0], dp, an[3:0]

local storage: d0, d1, d2, d3 (4 bits each), running (1 bit),

slow-clk (1 bit), sl-prev (1 bit), reset-clean (1 bit)



b. Processor architecture with datapath and controller FSM



c. Verilog, constraint, and bitstream

```
module StopwatchTimer(
    input clk,
    input reset,
    input start_stop,
    input [1:0] mode,
    input [7:0] time_value,
    output [6:0] sseg,
    output dp,
    output reg [3:0] an
);

// Slow clock
wire slow_clk;
clk_div cd(.clk(clk), .reset(reset_clean), .clk_out(slow_clk));

// Edge detection for start_stop and reset to fix debouncing
reg ss_prev;
reg running;
reg reset_prev;
reg reset_clean;
always @(posedge clk) begin
    ss_prev <= start_stop;
    if (reset_clean) begin
        running <= 0;
        ss_prev <= 0;
    end
    else if (start_stop && !ss_prev)
        running <= ~running;

    reset_prev <= reset;
    reset_clean <= reset && !reset_prev;
end

// Updating the digits
reg [3:0] d0, d1, d2, d3;

always @(posedge clk) begin
    if (reset_clean) begin
```

```

    case (mode)
        2'd0: begin d0<=0; d1<=0; d2<=0; d3<=0; end
        2'd1: begin d0<=9; d1<=9; d2<=9; d3<=9; end
        2'd2: begin d0<=0; d1<=0; d2<=(time_value[3:0]>9)? 9:time_value[3:0];
d3<=(time_value[7:4]>9)? 9:time_value[7:4]; end
        2'd3: begin d0<=0; d1<=0; d2<=(time_value[3:0]>9)? 9:time_value[3:0];
d3<=(time_value[7:4]>9)? 9:time_value[7:4]; end
        default: begin d0<=0; d1<=0; d2<=0; d3<=0; end
    endcase
end
else if (slow_clk && running) begin
    if (mode == 2'd0 || mode == 2'd2) begin
        if (d0==9 && d1==9 && d2==9 && d3==9) begin
            end
        else if (d0 == 9) begin
            d0 <= 0;
        if (d1 == 9) begin
            d1 <= 0;
        if (d2 == 9) begin
            d2 <= 0;
            d3 <= d3 + 1;
        end else d2 <= d2 + 1;
        end else d1 <= d1 + 1;
        end else d0 <= d0 + 1;
    end
else begin
    if (d0==0 && d1==0 && d2==0 && d3==0) begin
        end
    else if (d0 == 0) begin
        d0 <= 9;
    if (d1 == 0) begin
        d1 <= 9;
    if (d2 == 0) begin
        d2 <= 9;
        d3 <= d3 - 1;
    end else d2 <= d2 - 1;
    end else d1 <= d1 - 1;
    end else d0 <= d0 - 1;
    end
end
end

```

end

// Displaying the time

reg [1:0] sel;

reg [3:0] current_digit;

reg [16:0] display_clk;

always @(posedge clk) begin

if (reset_clean) begin

sel <= 0;

display_clk <= 0;

end else begin

if (display_clk == 99999) begin

display_clk <= 0;

sel <= sel + 1;

end else

display_clk <= display_clk + 1;

end

end

always @(*) begin

case (sel)

2'b00: begin an = 4'b1110; current_digit = d0; end

2'b01: begin an = 4'b1101; current_digit = d1; end

2'b10: begin an = 4'b1011; current_digit = d2; end

2'b11: begin an = 4'b0111; current_digit = d3; end

default: begin an = 4'b1111; current_digit = 4'b0; end

endcase

end

hexto7segment h(.x(current_digit), .r(sseg));

assign dp = (sel == 2'b10) ? 1'b0 : 1'b1;

endmodule

module hexto7segment(

input [3:0] x,

output reg [6:0] r

);

always@(*)

```

        case (x)
            4'b0000 : r = 7'b00000001; //0
            4'b0001 : r = 7'b10011111; //1
            4'b0010 : r = 7'b00100101; //2
            4'b0011 : r = 7'b00001110; //3
            4'b0100 : r = 7'b10011100; //4
            4'b0101 : r = 7'b01001100; //5
            4'b0110 : r = 7'b01000000; //6
            4'b0111 : r = 7'b00011111; //7
            4'b1000 : r = 7'b00000000; //8
            4'b1001 : r = 7'b00001100; //9
            4'b1010 : r = 7'b00010000; //A
            4'b1011 : r = 7'b11000000; //b
            4'b1100 : r = 7'b01100001; //C
            4'b1101 : r = 7'b10000101; //d
            4'b1110 : r = 7'b01100000; //E
            4'b1111 : r = 7'b01110000; //F
        endcase
    endmodule

```

```

module clk_div(
    input clk,
    input reset,
    output reg clk_out
);
    reg[19:0] COUNT;

    always @(posedge clk)
    begin
        if (reset) begin
            COUNT <= 0;
            clk_out <= 0;
        end
        else if (COUNT == 999999) begin
            COUNT <= 0;
            clk_out <= 1;
        end
        else begin
            COUNT <= COUNT +1;
            clk_out <= 0;
        end
    end

```

```
    end
end
endmodule
```

```
## Clock signal - Uncomment if needed (will be used in future labs)
set_property PACKAGE_PIN W5 [get_ports {clk}]
    set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
    create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clk}]
```

```
## Switches
```

```
set_property PACKAGE_PIN V17 [get_ports {time_value[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {time_value[0]}]
set_property PACKAGE_PIN V16 [get_ports {time_value[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {time_value[1]}]
set_property PACKAGE_PIN W16 [get_ports {time_value[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {time_value[2]}]
set_property PACKAGE_PIN W17 [get_ports {time_value[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {time_value[3]}]
set_property PACKAGE_PIN W15 [get_ports {time_value[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {time_value[4]}]
set_property PACKAGE_PIN V15 [get_ports {time_value[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {time_value[5]}]
set_property PACKAGE_PIN W14 [get_ports {time_value[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {time_value[6]}]
set_property PACKAGE_PIN W13 [get_ports {time_value[7]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {time_value[7]}]
set_property PACKAGE_PIN T1 [get_ports {mode[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {mode[0]}]
set_property PACKAGE_PIN R2 [get_ports {mode[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {mode[1]}]
```

```
## LEDs
```

```
set_property PACKAGE_PIN L1 [get_ports {slow_clk}]
    set_property IOSTANDARD LVCMOS33 [get_ports {slow_clk}]
```

```
##7 segment display
```

```
set_property PACKAGE_PIN W7 [get_ports {sseg[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sseg[6]}]
set_property PACKAGE_PIN W6 [get_ports {sseg[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sseg[5]}]
```

```
set_property PACKAGE_PIN U8 [get_ports {sseg[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sseg[4]}]
set_property PACKAGE_PIN V8 [get_ports {sseg[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sseg[3]}]
set_property PACKAGE_PIN U5 [get_ports {sseg[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sseg[2]}]
set_property PACKAGE_PIN V5 [get_ports {sseg[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sseg[1]}]
set_property PACKAGE_PIN U7 [get_ports {sseg[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {sseg[0]}]
set_property PACKAGE_PIN V7 [get_ports {dp}]
    set_property IOSTANDARD LVCMOS33 [get_ports {dp}]
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```

##Buttons

```
set_property PACKAGE_PIN U18 [get_ports {reset}]
    set_property IOSTANDARD LVCMOS33 [get_ports {reset}]
set_property PACKAGE_PIN T18 [get_ports {start_stop}]
    set_property IOSTANDARD LVCMOS33 [get_ports {start_stop}]
```